

Freight Rate Prediction from Tabular Shipment Data

Spotter ML Engineer assessment report

Prosper Arineitwe Asiimwe

Email: prox0591@gmail.com

Github Repository: <https://github.com/ProsperAsiimwe/spotter>

Live Demo of model prediction: <https://prosperasiimwe.dev/freight>

1 Introduction

This report answers the two main questions in the assessment:

1. how the labeled development data was split and validated (Section 2);
2. the December chart from the provided score.py (Section 3, Figure 2).

In this assignment, I predict `posted_rate` for truckloads. Labeled data is `data/train-test.csv`: 48,000 loads, January through October 2025. I score every row of `data/validation.csv` (12,000 November to December loads, no target) into `validation_predictions.csv` as `load_id,predicted_rate`. The models LightGBM (Ke et al., 2017), XGBoost (Chen and Guestrin, 2016), and CatBoost (Prokhorenkova et al., 2018) were used in this assignment. LightGBM had the lowest September to October Mean Absolute Error (MAE) at \$108.58. That model is retrained on all 48,000 labeled rows and used for the 12k file and the December chart.

I treat this as ordinary tabular regression, not a graph (GNN) or a per-lane time series. There are 10,793 origin-destination-equipment lanes, with a median of three loads each.

2 Train / test split and validation

The labeled file is split by calendar date. Rows are not shuffled.

January through August is training (38,477 loads). September through October is the holdout (9,523 loads). The holdout is only for comparing performance of the three models. The file we actually have to score is November to December, so this split fits well into that scenario. After LightGBM was chosen, I retrained it on all 48,000 labeled rows, then scored every row of `validation.csv`.

Fill-in values and city tables are fit on January-August only during that comparison. The random seed for reproducibility is 42.

`data/validation-predictions-template.csv` lists the same 12,000 ids, TE-000001 through TE-012000. The filled file is `validation_predictions.csv`.

Rates peak in June and sit lower in January (Figure 1). A random split would mix later months into training and would make the holdout look better than a true future-period test. This is why I didn't do random data splitting, but rather stuck to calendar date splits.

3 December prediction chart

Figure 2 is the chart from the provided `score.py`, run without changes, on `december-chart-inputs.csv`. Every row is Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Latitude, longitude, `market_index`, and `quote_signal` are not in the file.

Predicted rates run from \$834.98 to \$843.71. The labeled median on this Dry Van lane is \$808 ($n = 21$). Only the date changes, so the line is almost flat. The last days dip to about \$835 because of a calendar split, not because the model saw a crash. Christmas is tagged as a holiday and does not spike.

4 Exploratory Analysis

Distance is the main driver. Its correlation with `posted_rate` is 0.909. Median posted rate is about \$2,031 on a median haul of 953 miles. Dollars per mile still differ by trailer type, where Reefer \$2.31, Flatbed \$2.22, Dry Van \$2.05 (Figure 3). Weekends barely move the median (\$2,030 vs \$2,032). `quote_signal` times miles does not beat distance alone.

The labeled file has 64 cities; the scoring file has 72. Eight cities appear only in scoring: Allentown,

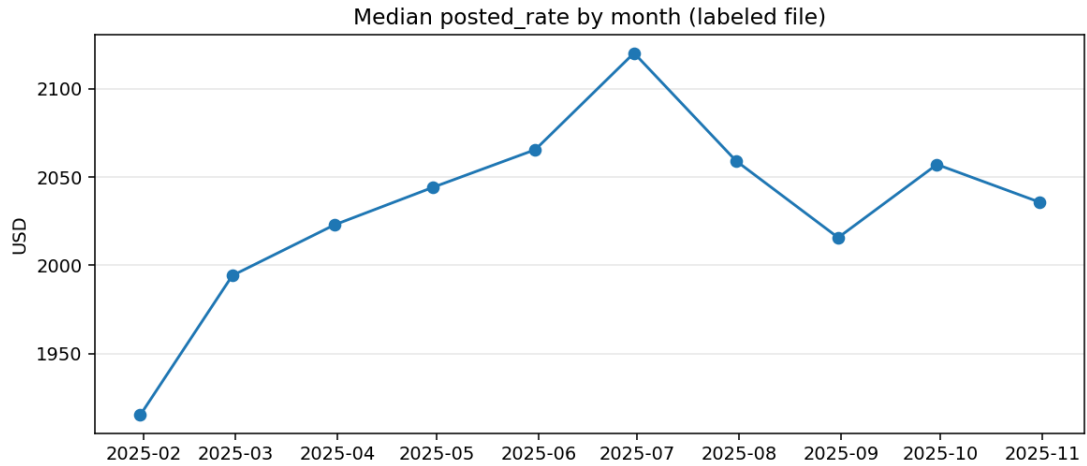


Figure 1: Median posted rate by month. September-October is the holdout. November-December are unlabeled.

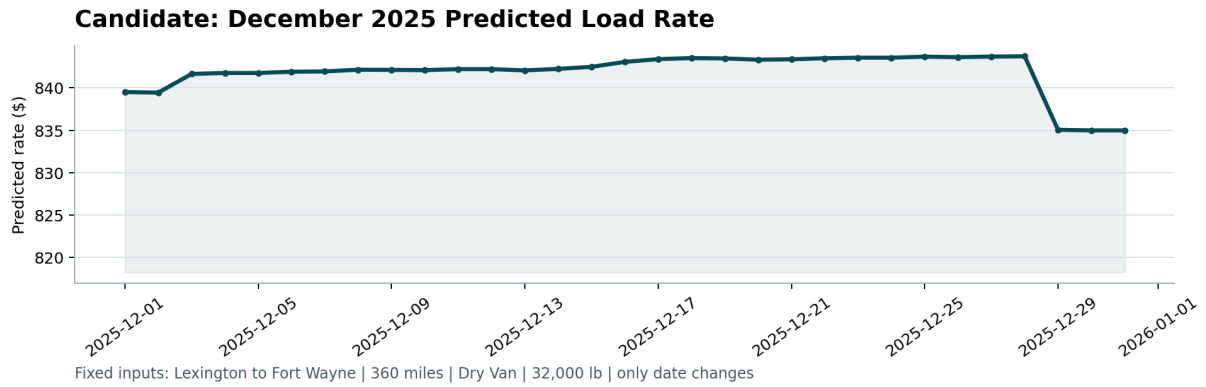


Figure 2: December chart from the provided score.py. Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb. Only the date changes.

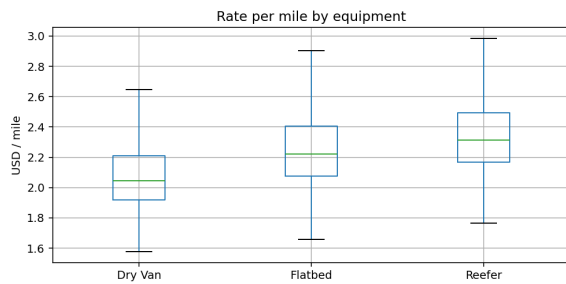


Figure 3: Median posted dollars per mile by equipment type.

Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, and San Diego.

5 Data Quality and Missing Values

Missing values show up in both files, so I cannot drop those rows.

Missing weight is heaviest on Dry Van (175 labeled rows). For weight, market_index,

Column	Labeled (48k)	Validation (12k)
weight	300	165
market_index	374	249

Table 1: Missing values in selected columns.

and quote_signal I add a yes-no column (weight_missing, market_missing, quote_missing) from the raw cell, then fill the number with the training median for that trailer type. The flag is 1 if the cell was empty, or if the column was not in the file, and 0 if a number was there. That way a filled-in typical weight is not treated as a measured one. Those medians are never taken from the holdout.

Unknown cities get an unknown code and an unseen-city flag. Missing coordinates use the training city table. In the December file, market and quote are missing entirely, so those two flags are all 1s.

There are no duplicate ids. Dates parse cleanly. Distance is clipped at one mile before the log. Predictions are clipped at \$0.01. The submitted 12k file ranges from \$222 to \$6,434, all positive, ids TE-000001 through TE-012000.

6 Features

src/freight/features.py builds the same 24 columns for the holdout, the 12k file, and December. After transformation, there are no NaNs: distance and log distance, equipment, weight with weight_missing, coordinates, city codes and unseen-city flags, calendar fields, and market and quote with their missing flags. Calendar fields are day of week, month, week, day of month, weekend, U.S. federal holiday, and days since 1 January 2025. Equipment, pickup, and delivery are categorical.

The target is $\log(1+y)$, inverted with $\exp(\hat{z}) - 1$ so a few large loads do not dominate training.

7 Models

Every model is scored on the same September-October holdout, using dollar MAE. I also report median absolute error, MAPE, and dollars-per-mile MAE.

Two baselines come first, i.e., median dollars per mile by equipment, times distance; and Ridge on $\log(1+y)$. Then LightGBM, XGBoost, and CatBoost on the same split. Trees fit because trailer type, cities, and month do not sit on a straight line. Table 2 is the September-October comparison.

Each tree family has its own ParameterGrid, scored by September-October MAE, not k -fold. A LightGBM DART run was much worse, so DART was dropped. The winner was LightGBM with learning rate 0.03, 63 leaves, at least 40 loads per leaf, column subsample 0.9, row subsample 0.7, $\lambda = 1$, and 409 trees. January-August MAE is \$75.56; September-October MAE is \$108.58.

Distance and log distance rank first in feature gain, then quote, pickup, delivery, equipment, weight, market, and calendar.

That winner is saved as models/rate_model.joblib and is what writes the 12k file and the December chart. The same artifact is on Hugging Face (<https://huggingface.co/Byteroot/lane-rate-lgbm>), with a FastAPI service at <https://huggingface.co/spaces/Byteroot/lane-rate>.

8 Conclusion

January-August trains, September-October selects, LightGBM at \$108.58 MAE is retrained on all 48,000 labeled rows, and that model scores the 12k November-December loads. The December line is not too messy because only the date changes. A random split would not be a better test of that job. The labeled window is only ten months, and many lanes are too thin for history features.

Reproducibility

Python 3.9 or later, from the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export PYTHONPATH=src
```

```
python3 scripts/eda.py
python3 scripts/baselines.py
python3 scripts/train.py
python3 scripts/predict.py
```

```
python3 score.py \
--predictions validation_predictions.csv \
--december-predictions december-chart-inputs.csv
```

PYTHONPATH=src is required so import freight works. On macOS, LightGBM needs OpenMP (brew install libomp if the import fails). score.py is used without changes. The December inputs sit in the repository root, not under data/.

References

- Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.
- Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.

Model	MAE (\$)	MedAE (\$)	MAPE (%)	RPM MAE
Equipment $\times$ miles	229.10	115.72	10.49	0.238
Ridge on $\log(1 + y)$	121.01	48.28	5.47	0.121
CatBoost	116.51	41.35	5.12	0.113
XGBoost	109.90	34.97	4.73	0.104
LightGBM	108.58	34.79	4.67	0.103

Table 2: September-October holdout. Lower is better.